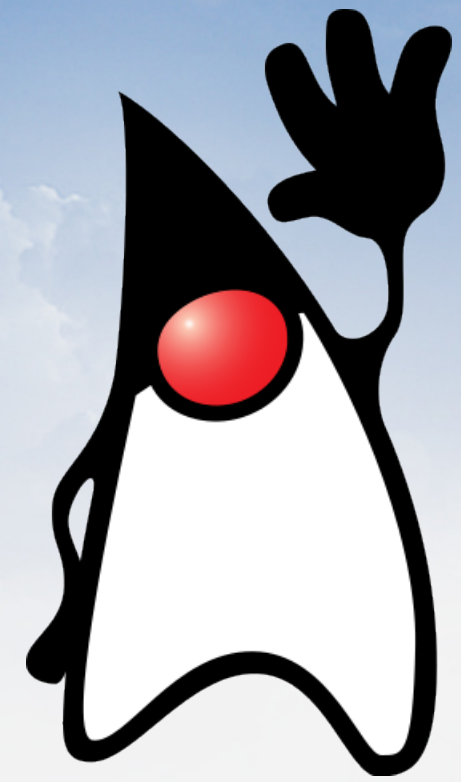
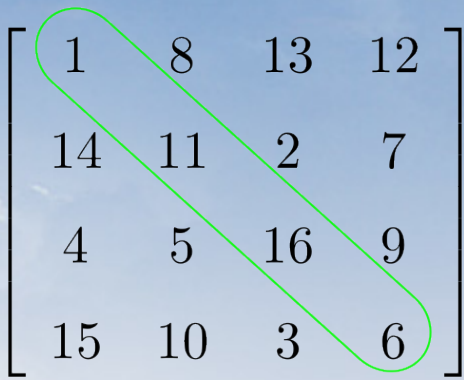




Run Time Analysis

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli



1	8	13	12
14	11	2	7
4	5	16	9
15	10	3	6

A question, which one is faster?

```
/*  
Fills the diagonal positions with 1  
@param int[][] arr with same row and  
column size  
*/  
public static void diagonalFill1(int[][] arr){  
    for(int i=0; i < arr.length; i++){  
        for(int j=0; j < arr[i].length; j++){  
            if( i == j ){  
                arr[i][j] = 1;  
            }//end of if  
        }//end of inner for j  
    }//end of outer for i  
}
```

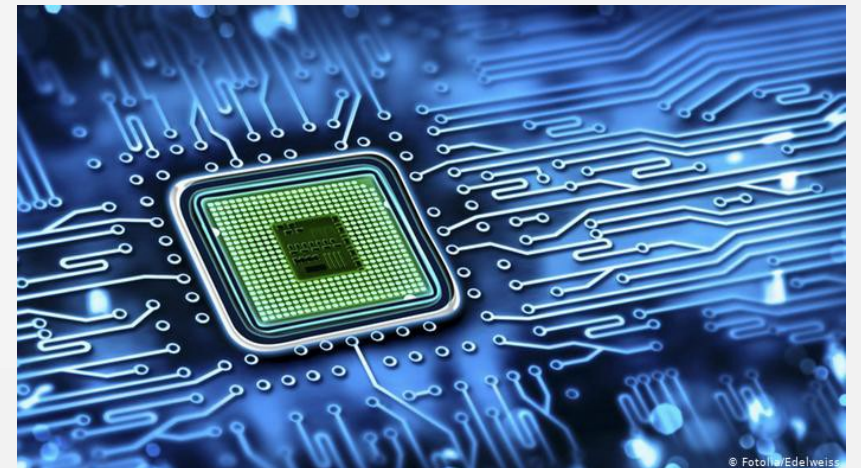
```
/*  
Fills the diagonal positions with 1  
@param int[][] arr with same row and  
column size  
*/  
public static void diagonalFill2(int[][] arr){  
    for(int i=0; i < arr.length; i++){  
        arr[i][i]=1;  
    }//end of outer for i  
}
```

Our Coding Solutions

- There exists a vast variety solutions to a single problem.
- Each has their own pros and cons
 - Running Time
 - Space Usage
- Today's problem in the world require critical thinking in how we can implement the best solution
 - Generally we try to find the fastest solution
 - Unless there is a space concern

Measuring our Runtime

- We must be careful with analyzing solutions.
- Cannot just say function foo takes 10 seconds!
 - Different Computers have different processing powers!
 - Architecture between systems is notably different.
- We need a universal way of expressing running time across all systems in general.



Best-Case, Worst-Case, Average-Case Runtimes

- There are 3 perspectives to analyze code
- Best-Case
 - Do not always argue the input the size of the data! This would translate into the worst-case
 - We must always assume an arbitrarily large input and then ask ourselves what conditions allow the function to terminate in the best possible time.
 - Example if we were to write a function to find an element in some large array, we could say that the element exists in the first index.
- Average-Case
 - This is common to observe from a practical perspective.
 - This is when input is randomly drawn from a given distribution.
- Worst-Case
 - The absolute worst possible scenario for a function to go through in solving a problem.
 - This is common for theoretical computer scientist to study.

Kinds of input

- Let's say for a integer sorting algorithm (ascending)
 - Best case: If the elements are sorted already (ascending)
 - Average case: The elements are random
 - Worst case: The elements are sorted already (descending)
- For each of them we specify a notation saying:
 - Best case: Omega (Ω) notation
 - The speed is not faster than $f(x)$, Ex: At best, This is the fastest it can be
 - Average case: Theta (Θ) notation
 - The speed is $f(x)$ Ex: On average, This is running exactly ...
 - Worst case: Big-Oh notation
 - The speed is not slower than $f(x)$ Ex: At worst, This is the slowest it can be

Big-Oh Notation

- Big Oh notation is shown as $O(f(x))$
- It means the set of functions (mathematical) whose speed of increase is less than or equal to $f(x)$
- Another way of saying mathematically, those functions are not increasing more than $f(x)$
- Programming wise, it means the running time of that algorithm is not greater than $f(x)$
- Another way of saying programmatically, that algorithm's running time is not worse than $f(x)$
- Perfect description to state the worst case scenario

Ω Omega Notation

- Omega notation is shown as $\Omega(f(x))$
- It means the set of functions (mathematical) whose speed of increase is greater than equal to $f(x)$
- Another way of saying mathematically, those functions are not increasing less than $f(x)$
- Programming wise, it means the running time of that algorithm is not less than $f(x)$
- Another way of saying programmatically, that algorithm's running time is not better than $f(x)$
- Perfect description to state the best case scenario

Θ Theta Notation

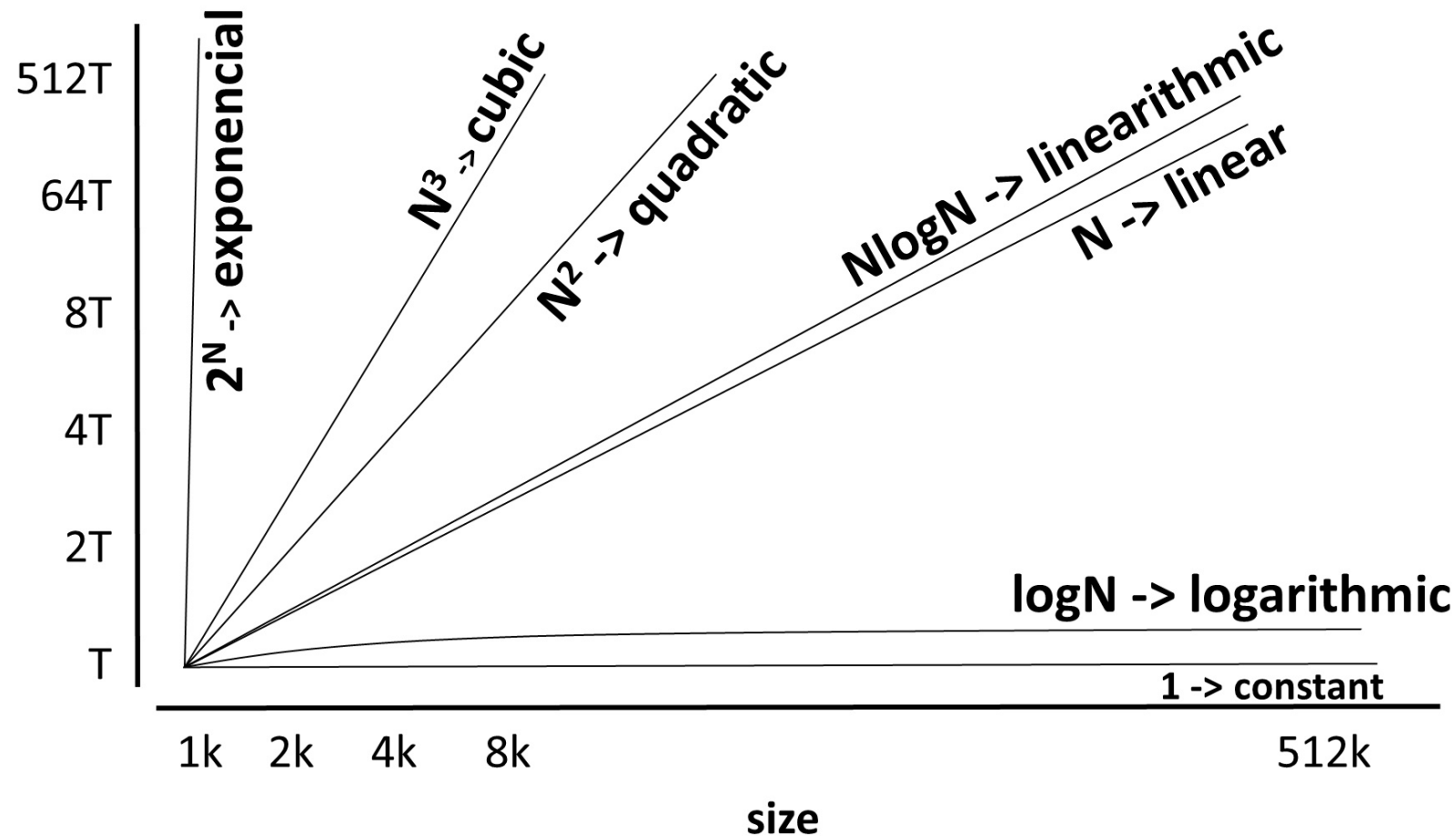
- Theta notation is shown as $\Theta(f(x))$
- It means the set of functions (mathematical) whose speed of increase is equal to $f(x)$
- Another way of saying mathematically, those functions are increasing similar with $f(x)$
- Programming wise, it means the running time of that algorithm is exactly $f(x)$
- Another way of saying programmatically, that algorithm's running time is exactly $f(x)$
- Perfect description to state the average case scenario

Actor functions in running time analysis



- 1 (meaning constant speed, not related with input size!)
- $\log n$ (increasing very slow)
 - If you are dividing the input every time then your running time is related with $\log n$
- n (linear time)
 - The speed is directly proportional with the input size
- n^2 (quadratic time)
 - The speed is proportional with square of the input size
- 2^n (Exponential time)
 - You are probably doing brute force search, listing subsets

Seeing the actors in a graph



$$\log_2 n = \log_{10} n \times \log_2 10$$

$\log_2 10$ is a constant
so there is not
much difference
between $\log_2 x$ and
 $\log_{10} x$, for very
large n .

Big Oh functions

- $O(n)$ covers those functions below
 - n , $n \times C$, C , 1
- $O(\log n)$ covers those functions below
 - $\log n$, $\log n \times C$, C , 1
- $O(n^2)$ covers those functions below
 - n^2 , $n^2 \times C$, $n \times C$, n , C , 1
- $O(2^n)$ covers those functions below
 - n^2 , n^3 , $\log n$, $n^4 \times n$
- $O(n!)$ covers all functions above

If $O(n!)$ covers all ,
why can't we use that
term to specify all
running complexities?

We need to specify
asymptotically tight
function for big-Oh
notation.

$O(n!)$ is not much
informative

Running Time with Multiple Variables

```
/*  
Fills the array with 1  
@param int[][] arr  
*/  
public static void diagonalFill1(int[][] arr){  
    for(int i=0; i < arr.length; i++){  
        for(int j=0; j < arr[i].length; j++){  
            arr[i][j] = 1;  
        }//end of inner for j  
    }//end of outer for i  
}
```

Here we have 2 input sizes row(n)
and column(m)

We will say the algorithm works in
 $O(n.m)$

will be done $n \times m$ times

**We can also say this is $O(n^2m^2)$
but it won't be informative**

Best Case

```
/*  
Fills the diagonal positions with 1  
@param int[][] arr  
*/  
public static void myFunc(int[] arr){  
    for(int i=0; i < arr.length; i++){  
        if(arr[i] % 2 ==0){ //even  
            for(int j=0; j< arr.length; j++){  
                arr[i]=2+arr[i]; //inner command  
            }  
        }  
    }  
} //end of outer for i  
}
```

How many times the inner command line will work at the best case?

Best case: is when all the elements are odd

$\Omega(n)$

$\Omega(n)$ means it can be n^2, n^3 but not 1 or $\log n$

Worst Case

```
/*  
Fills the diagonal positions with 1  
@param int[][] arr  
*/  
public static void myFunc(int[] arr){  
    for(int i=0; i < arr.length; i++){  
        if(arr[i] % 2 ==0){ //even  
            for(int j=0; j< arr.length; j++){  
                arr[i]=2+arr[i]; //inner command  
            }  
        }  
    }  
} //end of outer for i  
}
```

How many times the inner command line will work at the worst case?

Worst case: is when all the elements are even

$O(n^2)$

Average Case

```
/*  
Fills the diagonal positions with 1  
@param int[][] arr  
*/  
public static void myFunc(int[] arr){  
    for(int i=0; i < arr.length; i++){  
        if(arr[i] % 2 == 0){ //even  
            for(int j=0; j< arr.length; j++){  
                arr[i]=2+arr[i]; //inner command  
            }  
        }  
    }  
} //end of outer for i  
}
```

How many times the inner command line will work at the average case?

What is the probability that the if will be executed and not executed?

$$n \times \left(\begin{array}{cc} 0.5 \times 1 + & 0.5 \times n \\ \text{odd} & \text{even} \end{array} \right)$$

$$O(n \times (n+1) / 2) = O(n^2)$$

Binary Search

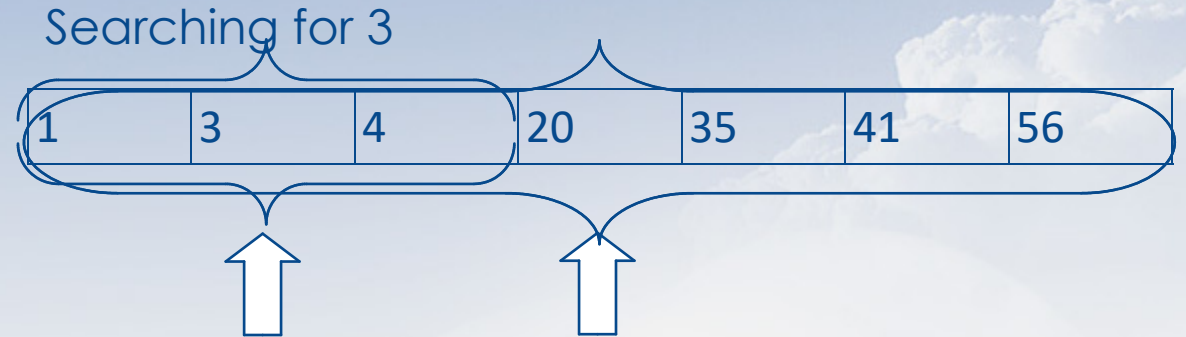
- An algorithm to search an array of sorted values
 - It divides the input by 2 in each iteration
 - The technique you use to find a page in a book
 - running time is $O(\log n)$
 - Even exists in Java as a method

What if the array is not sorted?

```
import java.util.Arrays;
public class arraysTest{
    public static void main(String args[])
    {
        int[] arr={3,1,5,2,7,6};
        Arrays.sort(arr);
        int searchedNum = 3;
        System.out.println( searchedNum + " found at index = "+
                            Arrays.binarySearch(intArr, intKey));
    }
}
```

Binary Search

```
public int bsearch(int[] arr, int el){
    int low=0;
    int high=arr.length;
    int mid=0;
    while(low < high){
        mid=(low+high) / 2 ;
        if(arr[mid] == el ){
            return mid; // position index
        }else if (arr[mid] < el ){
            low=mid+1;
            //we checked middle already
        }else{
            high = mid-1;
            //we checked middle already
        }
    }
    //if we came here
    //it means we couldn't find it
    return -1;
}
```



Best case: The element is in the middle, while will be done once. $\Omega(1)$

Worst case: The element does not exist, the input will be divided into half until 1 left.

How many times we can divide an number until we reach 1 or 0?

$\log_2 n$ times. ex: $\log_2 100 < 7$ (2^7)=128

Worst case running time is $O(\log n)$

Average case calculation of binary search

	Z		Y		Z		<u>X</u>		Z		Y		Z		
--	---	--	---	--	---	--	----------	--	---	--	---	--	---	--	--

$1/n \times 1$ comparison if the element is in X pos ($1/n$ possibility)

$2/n \times 2$ comparisons if the element is in Y positions ($2/n$ possibility)

$4/n \times 3$ comparisons if the element is in Z positions

$8/n \times 4 \dots$

we said we will be doing at most $\log n$ comparisons/divisions

so the summation becomes

$$\begin{aligned}
 & \left(\frac{1}{n} \times 1\right) + \left(\frac{2}{n} \times 2\right) + \left(\frac{4}{n} \times 3\right) + \left(\frac{8}{n} \times 4\right) + \dots = \\
 & \frac{1}{n} \times [(1 \times 1) + (2 \times 2) + (4 \times 3) + (8 \times 4) + \dots] = \\
 & \frac{1}{n} \times (1 \times 2^0) + (2 \times 2^1) + (3 \times 2^2) + \dots =
 \end{aligned}$$

$\log_2 n$ terms here

$$\begin{aligned}
 & \frac{1}{n} \times \left[\sum_{i=1}^{\log_2 n} i \times 2^{i-1} \right] = \\
 & \frac{1}{n} \times \left[\sum_{i=1}^{\log_2 n} (i \times 2^i) \times \frac{1}{2} \right] \approx \frac{1}{n} \times \left[[\log_2 n \times 2^{\log_2 n}] \times \frac{1}{2} \right] \approx \\
 & \frac{1}{2n} \times [\log_2 n \times n] \approx \log n
 \end{aligned}$$

Binary search remarks

Iterative or recursive implementations of the binary search has the same complexities.

Implementation type does NOT change the complexity.

Binary search needs sorted array to work.